SQL Part 1 – Terminology Concepts Reference

Database Systems

Based on Lecture 3 by Tiantian Liu

Fall 2025

Contents

1 Quick Reference Guide **3**

2 Introduction to SQL -Det virker **3**

3 SQL Syntax Basics **4**

4 SQL Data Types **4**

5 Creating Tables **5**

6 Integrity Constraints **5**

7 Modifying Tables **6**
- 7.1 INSERT . . . 6
- 7.2 DELETE . . . 7
- 7.3 DROP TABLE . . . 7
- 7.4 ALTER TABLE . . . 7

8 Basic Query Structure **8**

9 The SELECT Clause **8**
- 9.1 SELECT DISTINCT . . . 9
- 9.2 Arithmetic Expressions in SELECT . . . 10
- 9.3 SELECT * . . . 10

10 The WHERE Clause **10**
- 10.1 Logical Connectives . . . 11
- 10.2 Comparison Operators . . . 11
- 10.3 BETWEEN Operator . . . 12
- 10.4 NULL Values . . . 12

11 The FROM Clause & Multi-Table Queries **13**
- 11.1 Cartesian Product . . . 13
- 11.2 Joining Tables . . . 13

12 The RENAME Operation (AS) **14**

13 String Operations **15**
- 13.1 Case Conversion . . . 15
- 13.2 Pattern Matching with LIKE . . . 15

14 ORDER BY Clause **16**

15 Set Operations **17**

16 Aggregate Functions **19**
16.1 DISTINCT in Aggregates 20
16.2 NULL Handling 20

17 GROUP BY Clause **21**

18 HAVING Clause **22**

19 Query Execution Order **23**

20 Nested Subqueries **24**
20.1 Subqueries in WHERE 24
20.2 Subqueries in FROM 25
20.3 Subquery Tips 25

21 Common Mistakes **26**

22 Combining Operations - Complex Query Patterns **26**

23 Query Strategy Guide **28**

24 Summary **28**
24.1 SQL Clause Order 28
24.2 Essential Operations Quick Reference 29
24.3 Aggregate Functions 29
24.4 Key Reminders 29

1 Quick Reference Guide

Common Query Patterns - At a Glance

Basic Selection:

```
SELECT columns FROM table WHERE condition;
```

Join Two Tables:

```
SELECT cols FROM t1, t2 WHERE t1.key = t2.key;
```

Aggregation:

```
SELECT attr, COUNT(*) FROM table GROUP BY attr;
```

Filtered Aggregation:

```
SELECT attr, AVG(val) FROM table
WHERE condition GROUP BY attr HAVING AVG(val) > X;
```

Set Operations:

```
(SELECT ...) UNION/INTERSECT/EXCEPT (SELECT ...);
```

Key Execution Order

FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY

2 Introduction to SQL -Det virker

What is SQL?

SQL (Structured Query Language) is the standard language for relational database management systems.

- Widely used and well-supported
- Declarative language (specify *what* you want, not *how* to get it)
- Used for data definition, manipulation, and querying

SQL Parts

SQL consists of several sub-languages:

1. **DDL (Data Definition Language)**: Define schemas, create/modify/delete tables
2. **DML (Data Manipulation Language)**: Query and modify data
3. **Integrity**: Specify integrity constraints
4. **View Definition**: Define views
5. **Transaction Control**: Manage transactions
6. **Authorization**: Control access rights

3 SQL Syntax Basics

SQL Syntax Rules

- **Case-insensitive**: `SELECT` = `select` = `Select`
- **Exception**: String constants are case-sensitive: `'Fred'` ≠ `'FRED'`
- **String constants**: Use single quotes: `'abc'` (correct), `"abc"` (incorrect)
- **White-space**: Ignored (use for readability)
- **Statements**: End with semicolon (;)
- **Reserved words**: Cannot use as names (SELECT, FROM, WHERE, CREATE, etc.)

4 SQL Data Types

Common SQL Data Types

- **char(n)**: Fixed-length character string
  - Example: `char(5)` stores exactly 5 characters
- **varchar(n)**: Variable-length character string
  - Example: `varchar(20)` stores up to 20 characters
- **int**: Integer (machine-dependent range)
- **numeric(p,d)**: Fixed-point number
  - `p` = total digits, `d` = digits after decimal
  - Example: `numeric(3,1)` can store 44.5 but not 444.5 or 0.32
- **date**: Calendar date (year, month, day)
  - Format: `date '2018-04-25'`
- **time**: Time of day (hours, minutes, seconds)
  - Format: `time '09:30:00'`
- **timestamp**: Combination of date and time
  - Format: `timestamp '2018-04-25 10:29:01.45'`

5 Creating Tables

CREATE TABLE Syntax

```
CREATE TABLE table_name (
    attribute1 datatype1,
    attribute2 datatype2,
    ...,
    attributeN datatypeN,
    (integrity_constraint1),
    ...,
    (integrity_constraintK)
);
```

Simple CREATE TABLE

Create a table for instructors:

```
CREATE TABLE instructor (
    ID          char(5),
    name        varchar(20),
    dept_name   varchar(20),
    salary      numeric(8,2)
);
```

This creates a table with 4 columns:

ID	name	dept_name	salary
(empty)	(empty)	(empty)	(empty)

6 Integrity Constraints

Types of Integrity Constraints

1. **PRIMARY KEY**: Uniquely identifies each tuple
   - Implies NOT NULL and UNIQUE
2. **FOREIGN KEY**: References primary key of another table
   - Syntax: `FOREIGN KEY (attr) REFERENCES table(attr)`
3. **NOT NULL**: Attribute cannot be NULL
4. **UNIQUE**: All values must be distinct (NULL allowed)
5. **DEFAULT**: Provides default value if none specified

PRIMARY KEY - Method 1

Define primary key at the end of attributes:

```
CREATE TABLE instructor (
    ID          char(5),
```

```
    name        varchar(20) NOT NULL,
    dept_name   varchar(20),
    salary      numeric(8,2),
    PRIMARY KEY (ID),
    FOREIGN KEY (dept_name) REFERENCES department
);
```

PRIMARY KEY - Method 2

Define primary key inline with attribute:

```
CREATE TABLE instructor (
    ID          char(5) PRIMARY KEY,
    name        varchar(20) NOT NULL,
    dept_name   varchar(20) REFERENCES department,
    salary      numeric(8,2)
);
```

FOREIGN KEY with Explicit Reference

```
CREATE TABLE country (
    ID      char(5),
    name    varchar(20) UNIQUE NOT NULL,
    PRIMARY KEY (ID)
);

CREATE TABLE city (
    ID          char(5),
    name        varchar(20) NOT NULL,
    country_ID  char(5),
    PRIMARY KEY (ID),
    FOREIGN KEY (country_ID) REFERENCES country (ID)
);
```

DEFAULT Values

```
CREATE TABLE t1 (
    i       int DEFAULT -1,
    c       varchar(10) DEFAULT ' ',
    price   double(16,2) DEFAULT 0.00
);
```

7 Modifying Tables

7.1 INSERT

INSERT Syntax

```
INSERT INTO table_name VALUES (value1, value2, ..., valueN);
```

INSERT Example

```
INSERT INTO instructor VALUES
    ('10211', 'Smith', 'Biology', 66000);
```

Before:

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000

After:

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
10211	Smith	Biology	66000

7.2 DELETE

DELETE Syntax

```
DELETE FROM table_name;              -- Delete all tuples
DELETE FROM table_name WHERE condition;  -- Delete matching tuples
```

DELETE Example

```
DELETE FROM instructor;  -- Removes ALL tuples
```

7.3 DROP TABLE

DROP TABLE Syntax

```
DROP TABLE table_name;
```

Completely removes the table and all its data.

7.4 ALTER TABLE

ALTER TABLE Syntax

Add a column:

```
ALTER TABLE table_name ADD attribute datatype;
```

Drop a column:

```
ALTER TABLE table_name DROP attribute;
```

Note: Dropping attributes is not supported by many databases.

ALTER TABLE Example

Add a gender column:

```
ALTER TABLE instructor ADD gender char(1);
```

All existing tuples get NULL for the new attribute.

Before:

ID	name	salary
10101	Srinivasan	65000

After:

ID	name	salary	gender
10101	Srinivasan	65000	NULL

8 Basic Query Structure

Basic SQL Query

```
SELECT A1, A2, ..., An
FROM r1, r2, ..., rm
WHERE P;
```

- A_i = attributes to retrieve
- r_i = relations (tables) to query
- P = predicate (condition)

Maps to Relational Algebra:

$$\Pi_{A_1,A_2,\ldots,A_n}(\sigma_P(r_1 \times r_2 \times \ldots \times r_m))$$

Query Evaluation Steps

1. Compute Cartesian product of tables in FROM clause
2. Apply WHERE clause predicate
3. Project attributes from SELECT clause

Note: This describes the *logical* order, not how databases actually execute queries.

9 The SELECT Clause

SELECT Clause Purpose

The SELECT clause specifies which attributes (columns) to include in the query result.

- Corresponds to the **projection** (Π) in relational algebra

Basic SELECT

Find names of all instructors:

```
SELECT name
FROM instructor;
```

instructor table:

ID	name	dept_name
10101	Srinivasan	Comp. Sci.
12121	Wu	Finance
15151	Mozart	Music

Result:

name
Srinivasan
Wu
Mozart

9.1 SELECT DISTINCT

DISTINCT Keyword

By default, SQL does NOT eliminate duplicates. Use **DISTINCT** to remove duplicates.

```
SELECT DISTINCT attribute FROM table;
```

DISTINCT vs ALL

```
SELECT dept_name FROM instructor;           -- Includes duplicates
SELECT DISTINCT dept_name FROM instructor;  -- No duplicates
SELECT ALL dept_name FROM instructor;       -- Explicit: keep duplicates
```

instructor table:

name	dept_name
Srinivasan	Comp. Sci.
Wu	Finance
Katz	Comp. Sci.
Mozart	Music

SELECT dept_name:

dept_name
Comp. Sci.
Finance
Comp. Sci.
Music

SELECT DISTINCT dept_name:

dept_name
Comp. Sci.
Finance
Music

9.2 Arithmetic Expressions in SELECT

Arithmetic Operations

SELECT supports arithmetic: +, -, *, /

```
SELECT ID, name, salary/12
FROM instructor;
```

Arithmetic Example

Calculate monthly salary:

```
SELECT ID, name, salary/12 AS monthly_salary
FROM instructor;
```

instructor table:

ID	name	salary
10101	Srinivasan	65000
12121	Wu	90000

Result:

ID	name	monthly_salary
10101	Srinivasan	5416.67
12121	Wu	7500.00

9.3 SELECT *

SELECT * (All Attributes)

The asterisk (*) selects all attributes:

```
SELECT * FROM instructor;
```

Returns all columns from the instructor table.

10 The WHERE Clause

WHERE Clause Purpose

The WHERE clause filters tuples based on a predicate (condition).

- Corresponds to **selection** (σ) in relational algebra

Basic WHERE

Find instructors in Computer Science department:

```
SELECT name
FROM instructor
WHERE dept_name = 'Comp. Sci.';
```

instructor table:

ID	name	dept_name
10101	Srinivasan	Comp. Sci.
12121	Wu	Finance
45565	Katz	Comp. Sci.

Result:

name
Srinivasan
Katz

10.1 Logical Connectives

Logical Operators

Combine multiple conditions with:

- **AND**: Both conditions must be true
- **OR**: At least one condition must be true
- **NOT**: Negates a condition

Logical AND

Find Comp. Sci. instructors with salary > 70000:

```
SELECT name
FROM instructor
WHERE dept_name = 'Comp. Sci.' AND salary > 70000;
```

instructor table:

name	dept_name	salary
Srinivasan	Comp. Sci.	65000
Katz	Comp. Sci.	75000
Brandt	Comp. Sci.	92000

Result:

name
Katz
Brandt

10.2 Comparison Operators

Comparison Operators

- <: Less than
- <=: Less than or equal to
- >: Greater than
- >=: Greater than or equal to

- =: Equal to
- <>: Not equal to

10.3 BETWEEN Operator

BETWEEN Syntax

```
WHERE attribute BETWEEN value1 AND value2
```

Equivalent to: `attribute >= value1 AND attribute <= value2`

BETWEEN Example

Find instructors with salary between 90000 and 100000:

```
SELECT name
FROM instructor
WHERE salary BETWEEN 90000 AND 100000;
```

instructor table:

name	salary
Wu	90000
Einstein	95000
Gold	87000
Brandt	92000

Result:

name
Wu
Einstein
Brandt

10.4 NULL Values

NULL Handling

NULL represents unknown or missing values. Use special operators:

```
WHERE attribute IS NULL        -- Check if NULL
WHERE attribute IS NOT NULL    -- Check if not NULL
```

NEVER use: `= NULL` or `<> NULL` (these don't work!)

NULL Example

Find instructors without a department:

```
SELECT name
FROM instructor
WHERE dept_name IS NULL;
```

Find instructors with a department assigned:

```
SELECT name
FROM instructor
WHERE dept_name IS NOT NULL;
```

NULL in Comparisons

- Any comparison with NULL returns UNKNOWN (not TRUE or FALSE)
- `NULL = NULL` is UNKNOWN, not TRUE
- In WHERE: UNKNOWN is treated as FALSE
- Use `IS NULL` and `IS NOT NULL` for NULL checks

11 The FROM Clause & Multi-Table Queries

FROM Clause Purpose

The FROM clause lists the relations (tables) involved in the query.

- Corresponds to **Cartesian product** ($\times$) in relational algebra
- Use WHERE to create meaningful joins between tables

11.1 Cartesian Product

Cartesian Product

```
SELECT *
FROM instructor, teaches;
```

Generates every possible (instructor, teaches) pair.
Note: For common attributes (like ID), the result uses table.attribute naming:

- instructor.ID
- teaches.ID

11.2 Joining Tables

Join Pattern

To combine data from multiple tables, list tables in FROM and specify join condition in WHERE:

```
SELECT attributes
FROM table1, table2, ...
WHERE table1.key = table2.key AND other_conditions;
```

Two-Table Join

Find instructors who have taught courses:

```
SELECT name, course_id
```

```
FROM instructor, teaches
WHERE instructor.ID = teaches.ID;
```

instructor:

ID	name
10101	Srinivasan
15151	Mozart

teaches:

ID	course_id
10101	CS-101
10101	CS-315
15151	MU-199

Result:

name	course_id
Srinivasan	CS-101
Srinivasan	CS-315
Mozart	MU-199

Three-Table Join

Find names of instructors along with course titles they teach:

```
SELECT instructor.name, course.title
FROM instructor, teaches, course
WHERE instructor.ID = teaches.ID
  AND teaches.course_id = course.course_id;
```

Chain multiple tables by matching their common attributes.

Join Tips

- For n tables, you typically need $n - 1$ join conditions
- Use table aliases (AS) to simplify: `instructor AS i, teaches AS t`
- Always specify which table each attribute comes from if ambiguous
- Combine join conditions with AND in the WHERE clause

12 The RENAME Operation (AS)

AS Clause

The AS keyword renames relations and attributes:

`old_name AS new_name`

The keyword AS is optional: `old_name new_name`

Rename Relations

```
SELECT s.ID, d.ID
FROM department AS d, student AS s
WHERE d.ID = s.dept_ID;
```

Shorter table names make queries easier to read.

Rename Attributes

```
SELECT s.ID AS student_ID, d.ID AS department_ID
FROM department AS d, student AS s
WHERE d.ID = s.dept_ID;
```

Result columns are named:

student_ID	department_ID
...	...

13 String Operations

13.1 Case Conversion

String Functions

- **UPPER(s)**: Converts string to uppercase
- **LOWER(s)**: Converts string to lowercase
- **TRIM(s)**: Removes trailing spaces

13.2 Pattern Matching with LIKE

LIKE Operator

```
WHERE attribute LIKE pattern
```

Wildcards:

- %: Matches any substring (including empty)
- _: Matches any single character

LIKE Patterns

- `'Intro%'`: Matches strings starting with "Intro"
- `'%son'`: Matches strings ending with "son"
- `'%Comp%'`: Matches strings containing "Comp"
- `'___'`: Matches exactly 3 characters
- '

  ' :Matches3ormorecharacters
  %

LIKE Example

Find instructors whose name contains "dar":

```
SELECT name
FROM instructor
WHERE name LIKE '%dar%';
```

instructor:

name
Srinivasan
Sridharan
Mozart

Result:

name
Sridharan

Escaping Special Characters

To match literal % or _, use ESCAPE:

```
WHERE value LIKE '100\%' ESCAPE '\'
```

This matches the string "100%".

14 ORDER BY Clause

ORDER BY Syntax

```
SELECT ...
FROM ...
WHERE ...
ORDER BY attribute1 [ASC|DESC], attribute2 [ASC|DESC], ...;
```

- **ASC**: Ascending order (default)
- **DESC**: Descending order

ORDER BY Example

List instructors by name alphabetically:

```
SELECT name
FROM instructor
ORDER BY name;
```

Result:

name
Brandt
Califieri
Crick
Einstein
...(alphabetically)

ORDER BY DESC

List instructors by salary (highest first):

```
SELECT name, salary
FROM instructor
ORDER BY salary DESC;
```

Result:

name	salary
Einstein	95000
Wu	90000
Gold	87000
...	...

Multiple ORDER BY

Order by salary (DESC), then by name (ASC):

```
SELECT name, salary
FROM instructor
ORDER BY salary DESC, name;
```

For instructors with the same salary, results are sorted alphabetically by name.
Result:

name	salary
Einstein	95000
Wu	90000
Gold	87000
Brandt	65000
Mozart	65000
Srinivasan	65000

15 Set Operations

Set Operations

SQL supports three set operations:

- **UNION**: Combines results from two queries (removes duplicates)
- **INTERSECT**: Returns common tuples (removes duplicates)
- **EXCEPT**: Returns tuples in first query but not in second (removes duplicates)

To keep duplicates, use:

- UNION ALL
- INTERSECT ALL
- EXCEPT ALL

Union Compatibility

Set operations require **union-compatible** queries:

- Same number of attributes
- Compatible data types

Result schema uses column names from the first query.

UNION Example

Courses offered in Fall 2017 OR Spring 2018:

```
(SELECT course_id FROM section
 WHERE semester = 'Fall' AND year = 2017)
UNION
(SELECT course_id FROM section
 WHERE semester = 'Spring' AND year = 2018);
```

Fall 2017: CS-101, CS-347, PHY-101
Spring 2018: CS-101, CS-315, FIN-201, HIS-351
Result (UNION): CS-101, CS-315, CS-347, FIN-201, HIS-351, PHY-101

UNION ALL Example

Same query with UNION ALL (keeps duplicates):

```
(SELECT course_id FROM section
 WHERE semester = 'Fall' AND year = 2017)
UNION ALL
(SELECT course_id FROM section
 WHERE semester = 'Spring' AND year = 2018);
```

Result (UNION ALL): CS-101, CS-347, PHY-101, CS-101, CS-315, FIN-201, HIS-351
Note: CS-101 appears twice.

INTERSECT Example

Courses offered in Fall 2017 AND Spring 2018:

```
(SELECT course_id FROM section
 WHERE semester = 'Fall' AND year = 2017)
INTERSECT
(SELECT course_id FROM section
 WHERE semester = 'Spring' AND year = 2018);
```

Fall 2017: CS-101, CS-347, PHY-101
Spring 2018: CS-101, CS-315, FIN-201
Result (INTERSECT): CS-101

EXCEPT Example

Courses in Fall 2017 but NOT in Spring 2018:

```
(SELECT course_id FROM section
 WHERE semester = 'Fall' AND year = 2017)
EXCEPT
(SELECT course_id FROM section
 WHERE semester = 'Spring' AND year = 2018);
```

Fall 2017: CS-101, CS-347, PHY-101
Spring 2018: CS-101, CS-315, FIN-201
Result (EXCEPT): CS-347, PHY-101

16 Aggregate Functions

Aggregate Functions

Aggregate functions operate on a set of values and return a single value:

- **AVG(attr)**: Average value
- **MIN(attr)**: Minimum value
- **MAX(attr)**: Maximum value
- **SUM(attr)**: Sum of values
- **COUNT(attr)**: Number of values
- **COUNT(*)**: Number of tuples

AVG Example

Find average salary of CS instructors:

```
SELECT AVG(salary)
FROM instructor
WHERE dept_name = 'Comp. Sci.';
```

instructor (CS only):

name	salary
Srinivasan	65000
Katz	75000
Brandt	70000

Result: (65000 + 75000 + 70000) / 3 = 70000

COUNT Example

Count number of instructors:

```
SELECT COUNT(*) AS number
FROM instructor;
```

Result:

number
12

16.1 DISTINCT in Aggregates

DISTINCT vs ALL

- **ALL**: Include duplicates (default)
- **DISTINCT**: Remove duplicates before aggregation

Note: COUNT(*) does not support DISTINCT.

COUNT DISTINCT

Count number of different departments:

```
SELECT COUNT(DISTINCT dept_name) AS number
FROM instructor;
```

instructor:

dept_name
Comp. Sci.
Finance
Comp. Sci.
Music
Physics

Result: 4 (Comp. Sci., Finance, Music, Physics)

16.2 NULL Handling

NULL Values in Aggregates

- NULL values are **ignored** by aggregate functions (except COUNT(*))
- COUNT(*) counts all tuples, including those with NULL values
- COUNT(attr) counts only non-NULL values

NULL Example

```
SELECT COUNT(dept_name), COUNT(*)
FROM instructor;
```

instructor:

name	dept_name
Gold	NULL
Wu	Finance
Kim	NULL
Mozart	Music

Result:

COUNT(dept_name)	COUNT(*)
2	4

17 GROUP BY Clause

GROUP BY Purpose

GROUP BY partitions tuples into groups based on attribute values. Aggregate functions are then applied to each group separately.

```
SELECT attribute, AGG_FUNC(attribute)
FROM table
GROUP BY attribute;
```

GROUP BY Example

Find average salary per department:

```
SELECT dept_name, AVG(salary) AS avg_salary
FROM instructor
GROUP BY dept_name;
```

instructor:

dept_name	salary
Comp. Sci.	75000
Comp. Sci.	65000
Finance	90000
Finance	80000
Music	40000

Result:

dept_name	avg_salary
Comp. Sci.	70000
Finance	85000
Music	40000

GROUP BY with Multiple Attributes

Count courses per department per semester:

```
SELECT dept_name, semester, COUNT(*) AS course_count
FROM course, section
```

```
WHERE course.course_id = section.course_id
GROUP BY dept_name, semester;
```

Groups are formed for each unique combination of (dept_name, semester).

GROUP BY with Joins

You can combine GROUP BY with multi-table queries:

```
SELECT t1.attr, COUNT(t2.attr)
FROM table1 AS t1, table2 AS t2
WHERE t1.key = t2.key
GROUP BY t1.attr;
```

This counts occurrences from table2 for each group in table1.

18 HAVING Clause

HAVING Purpose

HAVING filters groups after GROUP BY is applied (like WHERE filters tuples).

```
SELECT attribute, AGG_FUNC(attribute)
FROM table
GROUP BY attribute
HAVING condition;
```

HAVING Example

Find departments with average salary > 42000:

```
SELECT dept_name, AVG(salary) AS avg_salary
FROM instructor
GROUP BY dept_name
HAVING AVG(salary) > 42000;
```

After GROUP BY:

dept_name	avg_salary
Biology	72000
Comp. Sci.	77333
Music	40000
Finance	85000

After HAVING:

dept_name	avg_salary
Biology	72000
Comp. Sci.	77333
Finance	85000

Music is excluded ($40000 \not> 42000$).

19 Query Execution Order

SQL Query Execution Sequence

1. **FROM**: Generate Cartesian product of tables
2. **WHERE**: Filter tuples based on condition
3. **GROUP BY**: Partition tuples into groups
4. **HAVING**: Filter groups based on aggregate condition
5. **SELECT**: Project specified attributes
6. **ORDER BY**: Sort result (if specified)

This is the *logical* order. Database optimizers may execute differently.

Complete Query Example

Find departments where instructors earning > 70000 have average salary > 75000:

```
SELECT dept_name, AVG(salary) AS avg_salary
FROM instructor
WHERE salary > 70000
GROUP BY dept_name
HAVING AVG(salary) > 75000;
```

Step 1 (WHERE): Filter salary > 70000

dept_name	salary
Finance	90000
Finance	80000
Physics	95000
Physics	87000
Comp. Sci.	75000
Comp. Sci.	92000

Step 2 (GROUP BY): Calculate averages

dept_name	avg_salary
Finance	85000
Physics	91000
Comp. Sci.	83500

Step 3 (HAVING): Filter avg > 75000 (all pass)
Final Result:

dept_name	avg_salary
Finance	85000
Physics	91000
Comp. Sci.	83500

20 Nested Subqueries

Subquery Basics

A subquery is a SELECT statement nested inside another query. Can be used in:

- WHERE clause: filter based on subquery results
- FROM clause: treat subquery result as a table
- SELECT clause: compute values (must return single value)

20.1 Subqueries in WHERE

IN Operator with Subquery

Find instructors who have taught courses:

```
SELECT name
FROM instructor
WHERE ID IN (SELECT ID FROM teaches);
```

The subquery returns a set of IDs, and IN checks membership.

NOT IN Operator

Find instructors who have never taught:

```
SELECT name
FROM instructor
WHERE ID NOT IN (SELECT ID FROM teaches);
```

Comparison Operators with Subqueries

- **SOME/ANY**: True if comparison is true for at least one value
- **ALL**: True if comparison is true for all values

```
WHERE salary > SOME (SELECT salary FROM ...)
WHERE salary > ALL (SELECT salary FROM ...)
```

Using SOME/ALL

Find instructors whose salary is greater than at least one Biology instructor:

```
SELECT name
FROM instructor
WHERE salary > SOME (SELECT salary
                     FROM instructor
                     WHERE dept_name = 'Biology');
```

Find instructors whose salary is greater than all Biology instructors:

```
SELECT name
FROM instructor
```

```
WHERE salary > ALL (SELECT salary
                    FROM instructor
                    WHERE dept_name = 'Biology');
```

EXISTS Operator

EXISTS returns TRUE if subquery returns at least one tuple:

```
WHERE EXISTS (SELECT * FROM ...)
WHERE NOT EXISTS (SELECT * FROM ...)
```

EXISTS Example

Find departments with at least one instructor:

```
SELECT dept_name
FROM department AS d
WHERE EXISTS (SELECT *
              FROM instructor AS i
              WHERE i.dept_name = d.dept_name);
```

Note: Subquery references outer query variable (correlated subquery).

20.2 Subqueries in FROM

FROM Subquery

Find departments with average salary above 42000:

```
SELECT dept_name, avg_salary
FROM (SELECT dept_name, AVG(salary) AS avg_salary
      FROM instructor
      GROUP BY dept_name) AS dept_avg
WHERE avg_salary > 42000;
```

The subquery is treated as a temporary table named `dept_avg`.

20.3 Subquery Tips

Subquery Best Practices

- Use IN/NOT IN for membership testing
- Use EXISTS/NOT EXISTS for checking existence (often faster)
- Use SOME/ALL for comparisons with multiple values
- Subqueries in FROM must have an alias
- Correlated subqueries reference outer query variables

21 Common Mistakes

SELECT with GROUP BY

When using GROUP BY, attributes in SELECT must be either:

- Listed in GROUP BY, OR
- Inside an aggregate function

Incorrect:

```
SELECT dept_name, name, AVG(salary)  -- name not in GROUP BY!
FROM instructor
GROUP BY dept_name;
```

Correct:

```
SELECT dept_name, AVG(salary)
FROM instructor
GROUP BY dept_name;
```

WHERE vs HAVING

- **WHERE**: Filters individual tuples (before grouping)
- **HAVING**: Filters groups (after grouping)

Incorrect:

```
SELECT dept_name
FROM instructor
GROUP BY dept_name
WHERE COUNT(*) > 2;  -- Cannot use WHERE here!
```

Correct:

```
SELECT dept_name
FROM instructor
GROUP BY dept_name
HAVING COUNT(*) > 2;
```

22 Combining Operations - Complex Query Patterns

Building Complex Queries

Complex queries often combine multiple concepts:

1. Multi-table joins (FROM multiple tables)
2. Filtering conditions (WHERE)
3. Grouping (GROUP BY)
4. Group filtering (HAVING)

5. Sorting results (ORDER BY)
6. Set operations (UNION, INTERSECT, EXCEPT)
7. Subqueries (nested SELECT)

Pattern: Join + Aggregate + Filter

Count courses taught by each instructor in CS department:

```
SELECT i.name, COUNT(t.course_id) AS num_courses
FROM instructor AS i, teaches AS t
WHERE i.ID = t.ID AND i.dept_name = 'Comp. Sci.'
GROUP BY i.ID, i.name;
```

Steps:

1. Join instructor and teaches tables
2. Filter for CS department
3. Group by instructor
4. Count courses per instructor

Pattern: Three Tables + Aggregate

Find total enrollment per department:

```
SELECT dept_name, COUNT(*) AS total_students
FROM course AS c, section AS s, takes AS t
WHERE c.course_id = s.course_id
  AND s.course_id = t.course_id
  AND s.sec_id = t.sec_id
  AND s.semester = t.semester
  AND s.year = t.year
GROUP BY dept_name;
```

Join three tables matching all relevant attributes.

Pattern: Set Operations with Conditions

Courses offered in Fall 2017 but not in Spring 2018:

```
(SELECT course_id FROM section
 WHERE semester = 'Fall' AND year = 2017)
EXCEPT
(SELECT course_id FROM section
 WHERE semester = 'Spring' AND year = 2018);
```

Pattern: Subquery + Join

Find instructors who teach in their own department's courses:

```
SELECT DISTINCT i.name
```

```
FROM instructor AS i, teaches AS t, course AS c
WHERE i.ID = t.ID
  AND t.course_id = c.course_id
  AND i.dept_name = c.dept_name;
```

23 Query Strategy Guide

Problem-Solving Approach

When writing queries:

1. **Identify tables**: Which tables contain the needed data?
2. **Determine joins**: How are tables related? What are join conditions?
3. **Filter conditions**: What WHERE conditions narrow down rows?
4. **Grouping needs**: Do you need to aggregate? By what attributes?
5. **Group filters**: Do you need HAVING to filter aggregated results?
6. **Output format**: What columns to SELECT? Need DISTINCT?
7. **Ordering**: Should results be sorted?

Common Query Patterns to Remember

- **Count per category**: `SELECT category, COUNT(*) FROM ... GROUP BY category`
- **Find maximum/minimum**: `SELECT MAX(attr) FROM ...` or use `ORDER BY ... LIMIT 1`
- **Entities with no relations**: Use `NOT IN` or `NOT EXISTS`
- **All entities in both sets**: Use `INTERSECT`
- **Entities above average**: Use subquery: `WHERE attr > (SELECT AVG(attr) FROM ...)`

24 Summary

24.1 SQL Clause Order

```
SELECT [DISTINCT] attribute_list
FROM table_list
WHERE condition
GROUP BY grouping_attributes
HAVING group_condition
ORDER BY sorting_attributes [ASC|DESC];
```

Execution Order: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY

24.2 Essential Operations Quick Reference

Operation	Syntax
Filter rows	`WHERE condition`
Join tables	`FROM t1, t2 WHERE t1.key = t2.key`
Remove duplicates	`SELECT DISTINCT ...`
Pattern match	`WHERE attr LIKE '%pattern%'`
Range check	`WHERE attr BETWEEN x AND y`
NULL check	`WHERE attr IS NULL / IS NOT NULL`
Count rows	`SELECT COUNT(*) FROM ...`
Group aggregate	`GROUP BY attr HAVING condition`
Set union	`(SELECT ...) UNION (SELECT ...)`
Set intersect	`(SELECT ...) INTERSECT (SELECT ...)`
Set difference	`(SELECT ...) EXCEPT (SELECT ...)`
Subquery membership	`WHERE attr IN (SELECT ...)`
Subquery existence	`WHERE EXISTS (SELECT ...)`
Subquery comparison	`WHERE attr > ALL (SELECT ...)`

24.3 Aggregate Functions

`AVG(attr)`, `MIN(attr)`, `MAX(attr)`, `SUM(attr)`, `COUNT(attr)`, `COUNT(*)`

Use `DISTINCT` inside aggregates to remove duplicates first: `COUNT(DISTINCT attr)`

24.4 Key Reminders

- Use `IS NULL` / `IS NOT NULL`, never `= NULL`
- For n tables, typically need $n - 1$ join conditions
- SELECT with GROUP BY: attributes must be in GROUP BY or inside aggregate
- WHERE filters rows before grouping; HAVING filters groups after
- Set operations require union-compatible queries (same # of columns, compatible types)
- String constants use single quotes: `'text'`
- Table/attribute aliases: `table AS alias` or just `table alias`